GSA

Application Areas
~~Phylogenetic Analysis~~

RNA / DNA 4 nucleotides. Sequences

Similar sequences → same or similar meaning / effect

Problems:

sequence comparison: compare two sequences and show the similarities.
eg: genes

sequence matching: find all positions in a sequence where a pattern sequence occurs
eg: AAT in genome

regular expression: find all positions in a text where a regular expression matches

multiple sequence matching: find all positions in a text where one of the sequences in a given set matches

approximate sequence matching: find all positions in a text where a sequence matches, allowing for errors in the match.

O - Notation

f is $O(g)$, if $0 \leq f(n) \leq c \cdot g(n)$, $\forall n \geq n_0 \in \mathbb{N}$

GSA

Sequence Comparision

$edist_S(u,v)$ can be computed in $O(m*n)$: Running Time

↳ It is a measure of local similarities in which matches between substrings are highly dependent on their relative positions in the sequence

↳ Not used: when this is not desired!

The sequence comparison problem

distant function δ

Hamming distance f_H:

$$f_h(u,v) = |\{i \mid 1 \le i \le n, u[i] \neq v[i]\}|$$

// only defined if $|u| = |v|$

The edit distance model

edit operation

edit operations are pairs $(\alpha, \beta) \in (A \cup \{\varepsilon\}) \times (A \cup \{\varepsilon\}) \setminus \{\varepsilon, \varepsilon\}$

$(\alpha \to \varepsilon)$: Deletion of char α
$(\varepsilon \to \beta)$: Insertion of char β
} Indels
$(\alpha \to \beta)$: Replacement of α with β

Sequence Comparison often also to represent the differences between two sequences as a collection of individual elementary differences
→ Alignment.

Aligment

alignment A is a sequence: $(\alpha_1 \to \beta_1, \dots, \alpha_h \to \beta_h)$

with $u = (\alpha_1, \dots \alpha_h)$, $v = (\beta_1, \dots, \beta_h)$

$$\begin{pmatrix} \alpha_1 & \dots & \alpha_h \\ \beta_1 & \dots & \beta_h \end{pmatrix}$$

GCSA

Edit Distance Model

Alignments

id = # of I's & D's & r = # of R's

Then

$$|u| + |v| = id + 2 \cdot r$$

Lemma: Alignment length ~ sequence length

u, v, $m = |u|$, $n = |v|$, $A = (\alpha_1 \to \beta_1, \ldots, \alpha_k \to \beta_q)$

$$\Rightarrow \quad m + n \geq q \geq \max\{m, n\}$$

The Number of Alignments

$Aligns(m,n)$: # of alignments of two fixed sequences with $|u| = m$ & $|v| = n$

~~Aligns~~ → only depends on m & n

$$Aligns(m,n) = \begin{cases} 1, & \text{if } m = 0 \text{ or } n = 0 \\ Aligns(m-1,n) + Aligns(m-1,n-1) + Aligns(m,n-1), & \text{else} \end{cases}$$

Approximated by Stanton-Cowan-Numbers:

$$Aligns(m,n) \approx (1+\sqrt{2})^{2n+1} \cdot \sqrt{n}$$

The number of subsequences

Order of D & I (hinkeimade) not important

↳ all alignments which differ only by the order of consecutive indels should be considered equivalent.

⇓

Subsequences

subsequence of u and v is a sequence of index pairs.

$$(i_1, j_1), \ldots, (i_r, j_r)$$

s.t:

$$1 \le i_1 < \ldots < i_r \le m \quad \&$$
$$1 \le j_1 < \ldots < j_r \le n$$

(i_k, j_k) stands for the replacement $u[i_k] \to v[j_k]$.

all other chars are considered to be deleted in u / v.

$\text{Subseq}(m, n)$ = # of subsequences of two sequences of length m & n

$$\text{Subseq}(m,n) = \sum_{r=0}^{\min(m,n)} \binom{m}{r} \cdot \binom{n}{r}$$

$$\text{Subseq}(m,n) \approx 2^{2n} \left(4 \cdot \sqrt{n\pi}\right)^{-1}$$

$\text{Align}(m,n) \gg \text{Subseq}(m,n)$ // Doesn't matter in reality!

GASA

The edit distance problem

Cost Function

Cost Function δ st. $\delta(\alpha \to \beta) \geq 0$ //R

$\delta(\alpha \to \beta) > 0$ //I, D

Symmetric iff: $\delta(\alpha \to \beta) = \delta(\beta \to \alpha)$

Zero property iff: $\delta(\alpha \to \beta) = 0$ iff $\alpha = \beta$

triangle inequality iff: $\delta(\alpha \to \beta) \leq \delta(\alpha \to \gamma) + \delta(\gamma \to \beta)$

$$\delta(A) = \sum_{i=1}^{n} \delta(\alpha_i \to \beta_i)$$

eg:

unit cost:
$$\delta(\alpha \to \beta) = \begin{cases} 0 & , \alpha = \beta, \alpha, \beta \in A \\ 1 & , \text{else} \end{cases}$$

Hamming cost:
$$\delta(\alpha \to \beta) = \begin{cases} 0 & , \alpha, \beta \in A, \alpha = \beta \\ 1 & , \alpha, \beta \in A, \alpha \neq \beta \\ \infty & , \text{else} \end{cases}$$

Edit Distance

$edist_\delta(u,v)$ is the minimum possible cost of an alignment of u and v:

$$edist_\delta(u,v) = \min\{\delta(A) \mid A \text{ is an alignment of } u \text{ \& } v\}$$

$\Leftrightarrow$ An Alignment A is optimal if $\delta(A) = edist_\delta(u,v)$

$\to$ unit edit distance if δ = unit cost function

$edist_\delta(u,v)$ is a metric on A^*

GrSA

Metric

$f : M \times M \to \mathbb{R}^{*+}$ is a metric if:

(1) $f(x,y) = 0 \iff x = y$ zero property

(2) $f(x,y) = f(y,x)$ symetric

(3) $f(x,y) \leq f(x,z) + f(z,y)$ triangle inequality

if (2) & (3) holds and $x = y \Rightarrow f(x,y) = 0 \to$ pseudo metric

if δ has (1) $\to edist_\delta$ has (1) if δ has (2) & (3), then $edist_\delta(u,v)$ is a metric on A^*

→ important: there can be more than one optimal Alignment

Levenshtein Distance = strict Edit Distance

Lemma

$\forall \delta$ and $\forall u,v \in A^*$:

$$edist_\delta(u,v) = edist_\delta(u^{-1}, v^{-1})$$

(u^{-1}, v^{-1}: reverse)

Def

edit distance problem: to compute the edit distance and all optimal alignments

GSA

Path

A path in a graph (V, E) is a sequence of nodes

$v_0, \dots, v_k$ s.t. $(v_i, v_{i+1}) \in E$

length k

$v_0 \to \dots \to v_k$ (directed)
$v_0 - \dots - v_k$ (undirected)

Weights

$g = (V, E)$ with weight function $w : E \to \mathbb{R}$ and path p:

$$w(p) = \sum_{i=0}^{k-1} w(v_i \to v_{i+1})$$

! interested in min path !

Problem

$G = (V, E)$ and $w : E \to \mathbb{R}$, let $s, x \in V$ and

$$\Delta(s, x) = \begin{cases} \min\{w(p) \mid p \text{ is a path from } s \text{ to } x\}, & \text{a path from } s \text{ to } x \text{ exists} \\ \infty, & \text{else} \end{cases}$$

Shortest-Path problem: for s and x consists of computing $\Delta(s, x)$ and a path p from s to x s.t. $w(p) = \Delta(s, x)$.

Negative Edges

eg:
in chemistry: compounds : nodes
reaction : edges
energy consumed or produced : edge label

Non-negative weights: shift weights by $-\min\{w(e) \mid e \in E\}$

Theorem

a shortest path from s to x which includes a path from x to y.
is a shortest path from x to y

GS★

Paths

For any nodes $s, u, x \in V$: $\Delta(s,u) \leq \Delta(s,x) + \Delta(x,u)$

Directed Acyclic graphs

$x \in V, \; s \prec x$

DAG

$$\Delta(s,x) = \min \{ \Delta(s,y) + w(y,x) \mid y \to x \in E \}$$

$O(|V| + |E|)$ running time!

Edit Graphs

edit graph $G(u,v)$ of u & v is an edge labeled directed graph. The nodes are the pairs (i,j), $0 \leq i \leq m, 0 \leq j \leq n$.

For $1 \leq i \leq m$, $0 \leq j \leq n$ deletion edge

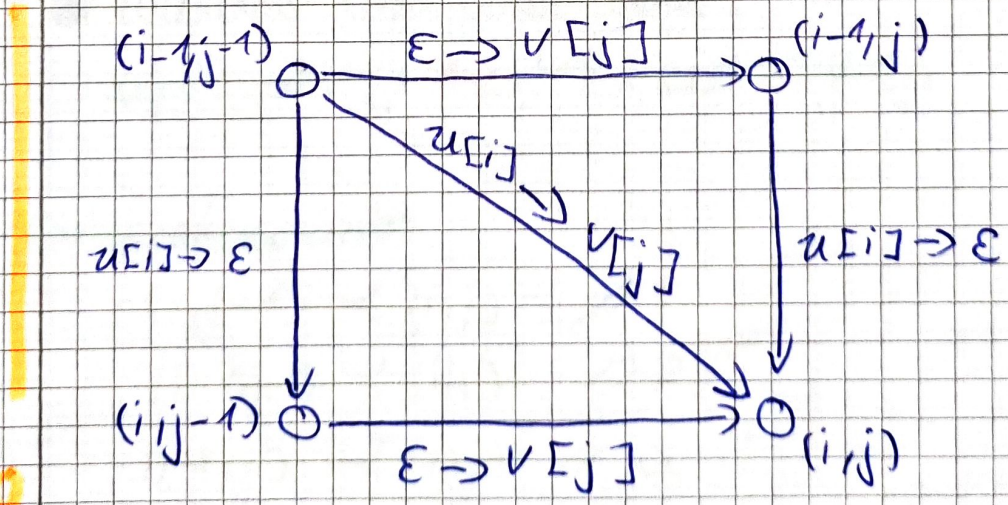
$$(i-1, j) \xrightarrow{u[i] \to \varepsilon} (i,j)$$

For $0 \leq i \leq m$, $1 \leq j \leq n$ insertion edge

$$(i, j-1) \xrightarrow{\varepsilon \to v[j]} (i,j)$$

For $1 \leq i \leq m$, $1 \leq j \leq n$ replacement edge

$$(i-1, j-1) \xrightarrow{u[i] \to v[j]} (i,j)$$



Each path from (i',j') to (i,j) is labeled by an alignment of $u[i'+1 \ldots i]$ & $v[j'+1 \ldots j]$

GSA

Edit Graphs

in edit graphs: path ~ alignment

$w((i',j') \xrightarrow{\alpha\to\beta} (i,j)) = S(\alpha \to \beta)$

shortest path = optimal alignment of $u[i'+1..i]$ & $v[j'+1..j]$

$(0,0) \to (m,n) \Rightarrow u[1..m]$ & $v[1..n]$

edist distance problem: shortest path problem in $G(u,v)$ $(0,0) \to (m,n)$

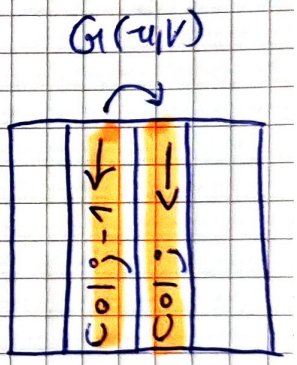
→ no cycles! ⇒ DAG

Topological Order

For a directed graph $G=(V,E)$ a topological order $\prec$ of V:

$$y \to x \in E \Rightarrow y \prec x$$

Two main topological orders in $G(u,v)$:



- columnwise topological order:

$(i',j') \prec (i,j) \Leftrightarrow j' < j$ or $(j'=j$ and $i'<i)$

- rowwise topological order:

$(i',j') \prec (i,j) \Leftrightarrow i' < i$ or $(i=i$ and $j' < j)$

in both cases:

$(i,j-1) \prec (i,j)$ // I

$(i-1,j-1) \prec (i,j)$ // R

$(i-1,j) \prec (i,j)$ // D

Edit Graph → Edit Matrices

$$E_\delta(i,j) = \Delta((0,0),(i,j))$$

$$E_\delta(i,j) = \min\{ E_\delta(i',j') + \delta(\alpha \to \beta) \mid (i',j') \xrightarrow{\alpha \to \beta} (i,j) \text{ is edge in } G(u,v) \}$$

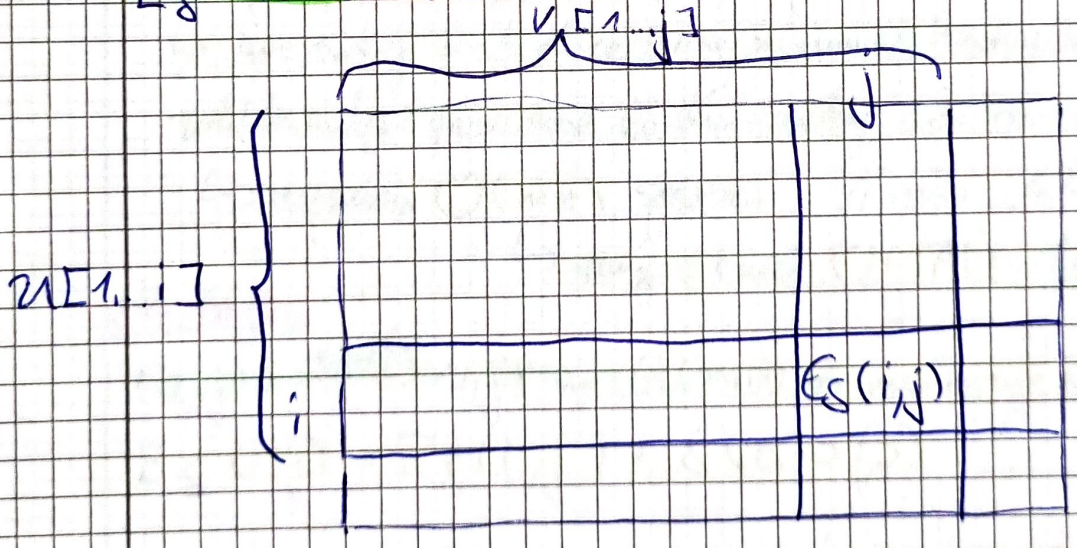
Recurrence:

$$E_\delta(i,j) = \begin{cases} 0 & , i=0, j=0 \\ E_\delta(0,j-1) + \delta(\varepsilon \to v[j]) & , i=0, j>0 \\ E_\delta(i-1,0) + \delta(u[i] \to \varepsilon) & , i>0, j=0 \\ \min \begin{cases} E_\delta(i-1,j) + \delta(u[i] \to \varepsilon) \\ E_\delta(i,j-1) + \delta(\varepsilon \to v[j]) \\ E_\delta(i-1,j-1) + \delta(u[i] \to v[j]) \end{cases} & , i>0, j>0 \end{cases}$$

$E_\delta(i,j)$ ⟺ weight of the shortest path from $(0,0)$ to (i,j) in $G(u,v)$

⟺ cost of an optimal alignment of $u[1..i]$ and $v[1..j]$

E_δ is a $(m+1) \times (n+1)$-matrix.



$$\Rightarrow E_\delta(m,n) = \text{edist}_\delta(u,v)$$

Algorithm: initialize row 0 & column 0

for all other recurrence! col-wise – or row-wise

GSA

Edit Distance Matrix

Each entry in E_δ is computed in constant time.
There are $(m+1)\cdot(n+1)$ entries => Running Time $(m\,n)$

-> $edist_\delta(u,v)$ only last value important

=> distance only algorithm: store just two columns.
↳ $O(m)$ space
"also possible one column + 2 scalars."

-> Dynamic Programing Algorithm.
↳ optimal solution can be derived from optimal solutions to subproblems
↳ the optimal solution can efficiently be determined, if a table of solutions for subproblems of increasing sizes are computed

-> Finding optimal alignment by tracing back from the entry $E_\delta(m,n)$ to an entry in its three-way minimum that yielded it, determining which entry gave rise to that entry, and so on till $E_\delta(0,0)$
↳ requires $O(m\,n)$ space. // needs whole E_δ
↳ runs time $O(|A|)$ traceback

Min Edge

$(i',j') \xrightarrow{\alpha\to\beta} (i,j)$ in $G_\delta(u,v)$ is minimizing edge if
$E_\delta(i,j) = E_\delta(i',j') + \delta(\alpha\to\beta)$

↳ Save min edge in bit vector while constructing E_δ

GASA

Edit Distance

edit distance problem for two sequences u, $|u| = m$, v, $|v| = n$, can be solved in $O(mn + z)$ time, where z is the total length of all alignments of u and v.

This method requires $O(mn)$ space.

Using distance-only algorithm in a recursive strategy, one obtains a divide and conquer algorithm that determine each optimal alignment in $O(\min\{m, n\})$ space and $O(mn)$ time.

GSA

Local similarity

↳ global alignment do not make sense.

DNA Sequences: often large non-coding regions and small coding regions.

two large sequences are similar $\not\Rightarrow$ small $edist_\gamma(u,v)$

Protein sequences often contain several domains which they share with related sequences. But if only the domains are similar, then a global comparison would not reveal their similarity.

Comparing biological sequences: it is sometimes important to perform local similarity comparisons:
find pairs of similar substrings of u and v

↳ It does not make sense to look for pairs of substrings with same minimum distance, as ε is a substring of any sequence and the sequences ε & ε have dist = 0 !

Score Function

score function σ assigns to each edit operation $\alpha \to \beta$ a score $\sigma(\alpha \to \beta) \in \mathbb{R}$.

For an Alignment A:
$$\sigma(A) = \sum_{i=1}^{k} \sigma(\alpha_i \to \beta_i)$$

The similarity score of the sequences u' and v' is defined by

$$score_\sigma(u',v') = \max\{\sigma(A) \mid A \text{ is an alignment of } u' \text{ and } v'\}$$

eg: Blosum 62 for comparing proteins

GSA

Local Similarity

Def

Let σ be a score function

1) $loc_\sigma(u,v) = \max\{score_\sigma(u',v') \mid u' \text{ is substring of } u \text{ and } v' \text{ is substring of } v\}$

↳ Max score over all pairs of substrings of u & v

2) Let u' substring of u and v' substring of v st:

$$score_\sigma(u',v') = loc_\sigma(u,v)$$

An alignment A of u' and v' satisfying $score_\sigma(u',v') = \sigma$ (*) is an optimal local alignment of u and v.

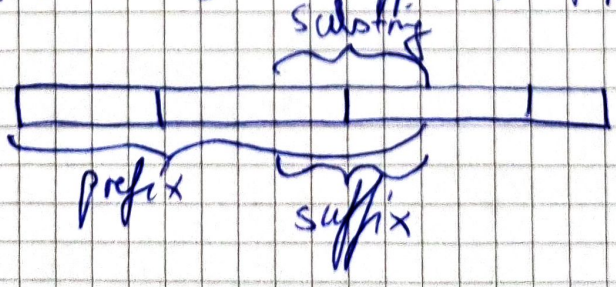
3) local alignment problem is to compute $loc_\sigma(u,v)$ and an optimal local alignment of u and v.

↳ brute force compute $score_\sigma(u',v')$ for all pairs of substrings u', v' of u, v. $O(m^3 n^3)$

- $score_\sigma(u',v')$ computes a global alignment problem, where similarities are maximized.
- $score_\sigma(u',v')$ can be computed in $O(mn)$ time

Note:

a substring of a string is a suffix of a prefix of that string



Local Alignments

Compute a matrix, each entry (i,j) contains the max score for all pairs of suffixes of prefix $u[1...i]$ of u and prefix $v[1..j]$ of v.

α_σ matrix $-(m+1)\times(n+1)$

$$\alpha_\sigma(i,j) = \max\{score_\sigma(x,y) \mid x \text{ is suffix of } u[1...i] \text{ & } y \text{ is suffix of } v[1..j]\}$$

~~$\alpha_\sigma(i,j)$ solves $loc_\sigma(u[1..i], v[1..j])$~~

$$loc_\sigma(u,v) = \max\{\alpha_\sigma(i,j) \mid 0 \le i \le m, 0 \le j \le n\}$$

Theorem

Let σ be a score function satisfying

(3.3) $loc_\sigma(s,\varepsilon) = loc_\sigma(\varepsilon,s) = 0$, $\forall s \in \Delta^*$

Then:

- $i=0$ & $j=0$, $\alpha_\sigma(i,j) = 0$
- else

$$\alpha_\sigma(i,j) = \max\begin{cases} 0 \\ \alpha_\sigma(i-1,j) + \sigma(u[i] \to \varepsilon) \\ \alpha_\sigma(i,j-1) + \sigma(\varepsilon \to v[j]) \\ \alpha_\sigma(i-1,j-1) + \sigma(u[i] \to v[j]) \end{cases}$$

! means that prefixes with negative score are suppressed, and many similar substrings occur as positive islands in a matrix dominated by 0-entries.

GSA

Local Alignment

Smith - Waterman Algorithm:

In: $u = [1..m]$, $v = v[1..n]$, score function σ satisfying (3.3)

Out: $\text{loc}_\sigma(u,v)$ and an optimal local alignment of u and v

1) Compute matrix $\mathcal{L}_\sigma$ with recurrence.
2) Compute a maximal entry, $\mathcal{L}_\sigma(i,j)$ in $\mathcal{L}_\sigma$
3) Compute optimal local alignments by a traceback from (i,j) on a maximizing path until some entry (i',j') with $\mathcal{L}_\sigma(i',j') = 0$.

Running Time: $O(mn)$ Space: $O(mn)$

↳ all pairs of substrings of both sequences are aligned to obtain an optimal local alignment

Alternatives Verfahren

Berechnet für jeden Knoten im Editgraphen $G(u,v)$ seinen Ursprung.

```
for i <- 0 to m
    for j <- 0 to n
        MO[i][j] <- calcMO(i,j)
    end
end
```

```
calcMO:
if loc(i,j) = 0
    return (i,j)
end
max (loc(i-1,j), loc(i,j-1), loc(i-1,j-1))
if max = loc(i-1,j) return
    calcMO(i-1,j-1)
⋮
```

GSA

Approximate String Matching Problem

Given pattern $p \in A^*$ of length m and an input string $t \in A^*$ of length n, ~~the approx~~

(APSMP) the approximate string matching problem = finding the positions in t, where an approximate match ends.

$k \in \mathbb{R}^+$ is a threshold value

$\Rightarrow$ approximate match is a substring w of t

$$edist_S(p, w) \leq k$$

$\Rightarrow$ semi global alignment!

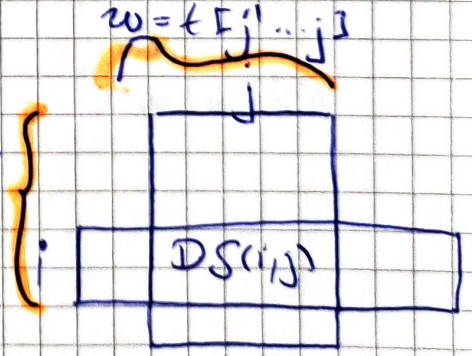
application:
when searching a DNA database for a pattern, few errors must be allowed. Small Differences between DNA of same species
APSMP is one of the most well-studied problems

Algorithm

compute $(m+1) \times (n+1)$-table D_S, s.t. $(i,j), 0 \leq i \leq m, 0 \leq j \leq n$:

$$D_S(i,j) = \min\{edist_S(p[1...i], w) \mid w \text{ is a suffix of } t[1...j]\}$$

$\rightarrow$ an approximate match ending at position j, $0 \leq j \leq n$, iff $edist_S(p, w) \leq k$ for suffix w of $t[1...j]$



$\Leftrightarrow D_S(m,j) \leq k$

$\Rightarrow$ Solving APSMP: Compute D_S and output all j with $D_S(m,j) \leq k$

GSA

Approximate String Matching Problem

Approximate matches of p ending at position j and an optimal alignment of p and w. computed via $\min$ traceback from $D_\delta(m,j)$ to an entry $D_\delta(0,j')$, $0 \leq j' \leq j-1$

$$D_\delta(i,j) = \begin{cases} 0 & , i=0 \\ D(i-1,0) + \delta(p[i] \to \varepsilon) & , i>0 \wedge j=0 \\ \min \begin{cases} D_\delta(i-1,j) + \delta(p[i] \to \varepsilon) \\ D_\delta(i,j-1) + \delta(\varepsilon \to t[j]) \\ D_\delta(i-1,j-1) + \delta(p[i] \to t[j]) \end{cases} & , i>0 \wedge j>0 \end{cases}$$

Computation like E_δ.

Computation of D_δ Runtime: $O(mn)$

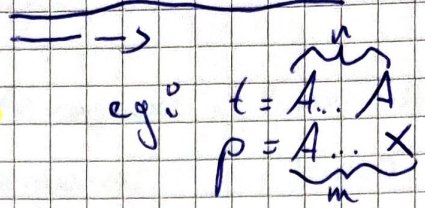
When only the positions are considered then Space $O(m)$

- Compute approximate matches using traceback of minimizing paths one does not have to store the entire table D_δ
  - ↳ k determines an upper bound for # of ~~edits~~ $I \triangleq R$.
  - ↳ a minimizing path from $D_\delta(m,j)$ to $D_\delta(0,j')$ has max length of $m+\gamma$, $\gamma \leq \left\lfloor \frac{k}{\min\{\delta(\varepsilon \to b) \mid b \in A\}} \right\rfloor$
  - ↳ Deletions here do not affect the ~~distance~~ length of a match ($\to R$)
  - ↳ Space requirement $O(m(m+\gamma))$ (γ: I)
  - ↳ For the unit cost function. $\gamma = k \to O(m(m+k))$

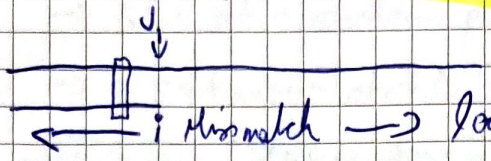
GSA

Exaktes String Matching

Naive Algorithmus $\mathcal{O}(nm)$

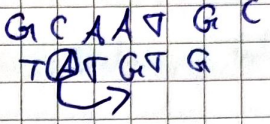


Better with bad character table!



mismatch → look bad_char_table up and increment j by this value.

Compute Bad Char Table:

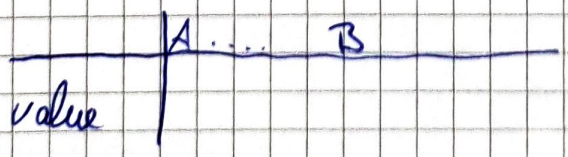



```
compute(p, t):
    ∀ c ∈ A:
        ∀ i in len(p) - 1:
            if p[i] = c
                bad-table[c] = len(p) - i - 1
        if c not in bad_table:
            bad-table[c] = len(p)
```

save the right most occurrence!

max(1, length - index - 1)

↳ Für ein Alphabet der Größe t:



text[j .. j+m-1]

← wie weit der Character p[m-1] von dem current char entfernt ist.

GSA

Overlap problem in read assembly

Sequencing DNA: determine the sequence of bases, the DNA consist of.
↳ target sequence 30000 - Million Bases

↳ One need to apply shotgun sequencing technique. Sequences reads
→ start at a random position in one of the two strands
→ a continuous stretch of bases is sequenced, 5' to 3'

↳ Outputs many diff reads. Which direction read / start position of read

⇒ read assembly problem: determining the entire target sequence, by assembling the given reads in the correct orientation and order

Phases

→ Overlap phase: Determine suffix/prefix overlaps for all pairs of reads.

→ Layout phase: Determine a multiple Alignment, derived from the positioning of the different reads above each other, and from the introduction of gap symbols

→ Consensus phase: determine the target sequence by considering the diffent columns of the multiple alignment.

No Exact Reads

$0 \leq \epsilon \leq 1$

ϵ = # of errors per base of read

→ Compute approximate overlaps.
↳ For reads of the same length m, we allow $2m\epsilon$ errors.

GrSA

Read Assembly

Overlap

alignment A is an overlap of f and f', if there are substrings u of f, v of f', A is an alignment of u & v with (at least one)°

1) u is suffix of f and v is a prefix of f'
2) u is prefix of f and v is a suffix of f'
3) $u = f$ and v is a substring of f'
4) u is a substring of f and $v = f'$

$\text{len}(A) = (|u| + |v|) / 2$

Two reads f'/f overlap with an error rate: $\delta(A) \leq k = \varepsilon(|f| + |f'|)$

⇒ Overlap problem for a pair f and f' of reads°:
deciding whether f and f' overlap and if so to compute an overlap A of f and f' with $\delta(A) \leq k$

CoSA

Overlap f, m, f', n

Compute $(m+1) \times (n+1)$ - table Ov_δ s.t. $(i,j), 0 \le i \le m, 0 \le j \le n$:

$$Ov_\delta(i,j) = \min\{ edist_\delta(u,v) \mid u \text{ is a suffix of } f[1\ldots i] \text{ and } v = f'[1\ldots j] \text{ or } u = f[1\ldots i] \text{ and } v \text{ is a suffix of } f'[1\ldots j] \}$$

There is an overlap of f and f', iff:

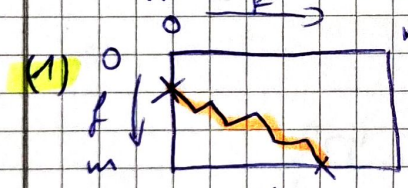
1) $Ov_\delta(i,n) \le k$, $0 \le i \le n$ or
2) $Ov_\delta(m,j) \le k$, $0 \le j \le m$

Prefix / Suffix can use indels with no costs.

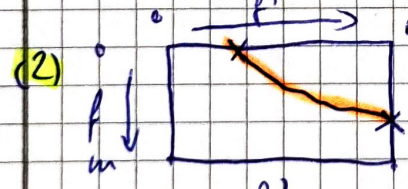
$\Rightarrow$

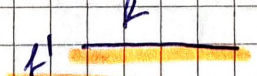
$$Ov_\delta(i,j) = \begin{cases} 0 & , i=0 \text{ or } j=0 \\ \min\begin{cases} Ov_\delta(i-1,j) + \delta(f[i] \to \varepsilon) \\ Ov_\delta(i,j-1) + \delta(\varepsilon \to f'[j]) \\ Ov_\delta(i-1,j-1) + \delta(f[i] \to f'[j]) \end{cases} & , \text{else} \end{cases}$$

Four Types Of Paths

(1) 

start: $(i',0)$ end: (m,j)

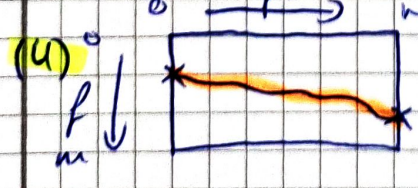
(2) 



start: $(0,j')$ end: (i,n)

(3)

start: $(0,j')$ end: (m,j)

(4) 

start: $(i',0)$ end: (i,n)

Ex 4 A

Overlap

Determine $len(A)$:

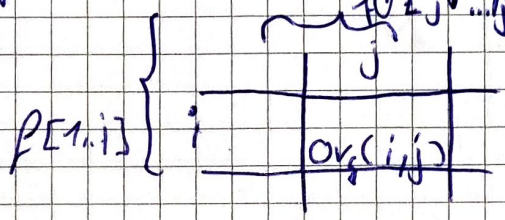
→ $\forall$ overlap starts $(i', 0)$ one track the row from which it starts, minimizing the row number if in the edit graph there is more than one minimizing edge into the current node.

→ vice versa for rows.

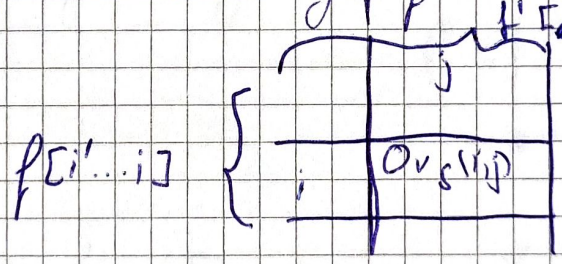
minimizing choice ⟶ maximizes the overlap length.

⇒ $len(A) \geq \tau$

A prefix of f matches a substring of f': start $(0, j)$



A prefix of f' matches a substring of f: start $(i', 0)$



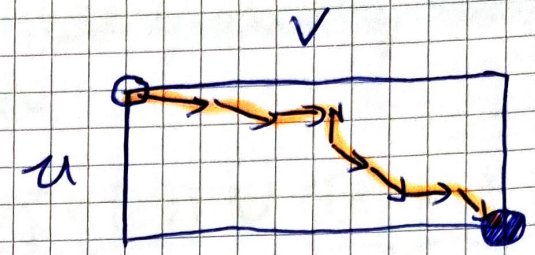
GSA

Summary Pairwise Comparison

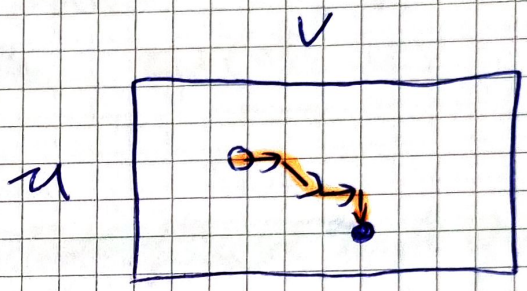
Sequence Comparison Methods so far computes $(m+1) \times (n+1)$ - table in $O(mn)$ time.
Local alignment maximizes score while others minimize distances.

Traceback

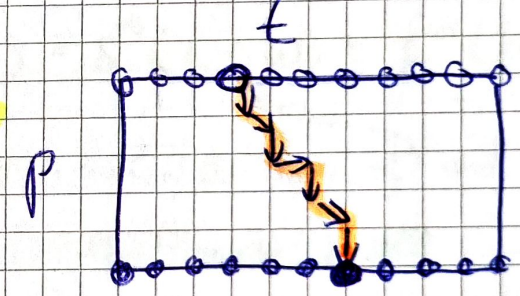
global alignment (E_S)



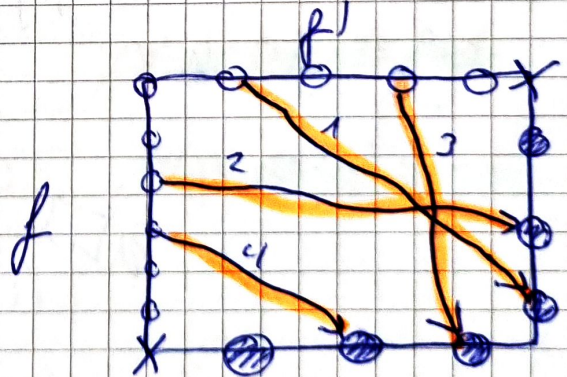
local alignment (α_S)



Approximate String Searching (D_S)



Fragment Overlap (Ov_S)



one of 1, 2, 3, 4

GSA

Variations of the cost and score model

Usual evolutionary programm of biosequences involves individual point mutations: R, D, I of a single nucleotide / residue.
Sometimes: Deletion / Insertion of entire segments as single unit.

↳ **Gaps:** extend the alignment model by intervals of unaligned (indels) characters.

$$(\alpha,\beta) \in (A \times A) \cup \underbrace{(A^+ \times \{\varepsilon\})}_{|gap| \geq 1} \cup \underbrace{(\{\varepsilon\} \times A^+)}_{|gap| \geq 1}$$

for each edit operation $\alpha \to \beta$ eg: $(abc \to \varepsilon)$

General Gap Costs

Costs by an arbitrary computable function f^* on the length of the gap.

↳ $w \in A^+$:
$$\delta^*(w \to \varepsilon) = \delta^*(\varepsilon \to w) = f^*(|w|)$$

cost for an alignment: $\delta^*(A)$; sum of the costs of used R's

↳ Extend Edit Graph $G(u,v)$, (i,j), $0 \leq i \leq m$, $0 \leq j \leq n$:

$$(i',j) \xrightarrow{u[i'+1 \ldots i] \to \varepsilon} (i,j) \quad , \forall i', 0 \leq i' \leq i-1$$

&

$$(i,j') \xrightarrow{\varepsilon \to v[j'+1 \ldots j]} (i,j) \quad , \forall j', 0 \leq j' \leq j-1$$

$\Rightarrow$ # of edges $\mathcal{O}(mn(m+n))$

General Gap Costs

edit

compute $(m+1) \times (n+1)$ - table GGC_{δ,f^*}

$GGC_{\delta,f^*}(i,j)$: is the generalized edit distance of $u[1..i]$ & $v[1..j]$ with respect to the cost functions δ and f^*.

$$GGC_{\delta,f^*}(i,j) = \begin{cases} 0 & , i=j=0 \\ \min\{GGC_{\delta,f^*}(i',j) + f^*(i-i') \mid 0 \le i' \le i-1\} & , i>0 \\ \min\{GGC_{\delta,f^*}(i,j') + f^*(j-j') \mid 0 \le j' \le j-1\} & , j>0 \\ GGC_{\delta,f^*}(i-1,j-1) + \delta(u[i] \to v[j]) & , i,j>0 \end{cases}$$

each entry computed in $O(m+n)$

↳ Total Runtime $O(mn(m+n))$ // Needle Man-Wunsch Alg

Affine Gap Costs

Each gap gets the sum of the costs for each symbol involved in the gap plus a gap initiation cost.

$w \in A^+$: $\delta_{affine}(w \to \varepsilon) = g + \sum_{i=1}^{|w|} \delta(w[i] \to \varepsilon)$

&

$\delta_{affine}(\varepsilon \to w) = g + \sum_{i=1}^{|w|} \delta(\varepsilon \to w[i])$

$\delta_{affine}(A)$

Edit Distance Problem with affine gap costs:

finding alignment A of u and v s.t. $\delta_{affine}(A)$ is minimal

Affine Gap Costs

Affine gap costs provide incentive for alignment algorithm to keep gaps together where possible rather than scattering many small gaps.

- mark which edge starts/ends a gap in $G_A(u,v)$
- 3 different nodes (i,j,R), (i,j,D), (i,j,I)
-

↳ replacement edge:

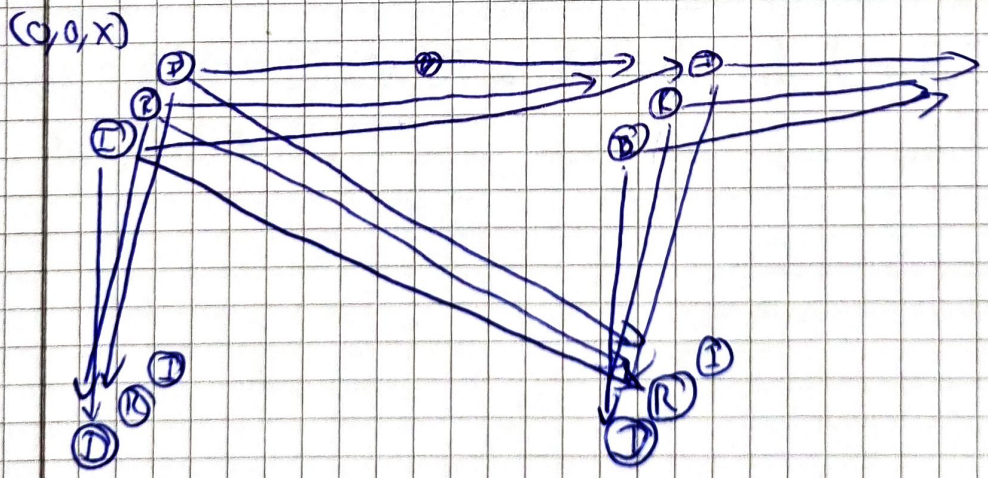
$$(i-1, j-1, X) \xrightarrow{u[i] \to v[j]} (i,j,R) \quad , X \in \{R,D,I\}$$

↳ insertion edge:

$$(i, j-1, X) \xrightarrow{\varepsilon \to v[j]} (i,j,I) \quad , X \in \{R,D,I\}$$

↳ deletion edge:

$$(i-1, j, X) \xrightarrow{u[i] \to \varepsilon} (i,j,D) \quad , X \in \{R,D,I\}$$



GSA

Affine Gap Costs

Costs:

Kind of edge	edit operation	costs
$(i-1, j-1, x) \xrightarrow{u[i] \to v[j]} (i, j, R)$	Replacement	$\delta(u[i] \to v[j])$
$(i-1, j, D) \xrightarrow{u[i] \to \varepsilon} (i, j, D)$	gap extension	$\delta(u[i] \to \varepsilon)$
$(i, j-1, I) \xrightarrow{\varepsilon \to v[j]} (i, j, I)$	gap extension	$\delta(\varepsilon \to v[j])$
$(i-1, j, R) \xrightarrow{u[i] \to \varepsilon} (i, j, D)$	gap start	$g + \delta(u[i] \to \varepsilon)$
$(i-1, j, I) \xrightarrow{u[i] \to \varepsilon} (i, j, D)$	gap start	$g + \delta(u[i] \to \varepsilon)$
$(i, j-1, R) \xrightarrow{\varepsilon \to v[j]} (i, j, I)$	gap start	$g + \delta(\varepsilon \to v[j])$
$(i, j-1, D) \xrightarrow{\varepsilon \to v[j]} (i, j, I)$	gap start	$g + \delta(\varepsilon \to v[j])$

A path $(i', j', x) \xrightarrow{*} (i, j, y)$ corresponds to an alignment of $u[i'+1..i]$ & $v[j'+1..j]$, that ends with edit operation y.

$(0,0,R), (0,0,D), (0,0,I)$ as start entry and $(m,n,R), (m,n,D), (m,n,I)$ as final entries.

GST

Affine Gap Costs

compute $(m+1) \times (n+1) \times \{R, D, I\}$ - table A_{afn} s.t.

$i = j = 0$: $A_{afn}(i,j,R) = 0$ and $A_{afn}(i,j,y) = g$, $y \in \{D, I\}$

$i > 0$ or $j > 0$:

$$A_{afn}(i,j,R) = \begin{cases} \infty & , i = 0 \text{ or } j = 0 \\ \min\{A_{afn}(i-1,j-1,y) + S(x[i] \to v[j]) \mid y \in \{R,D,I\}\} & , \text{else} \end{cases}$$

~~$A_{afn}(i,j,D) = \min\{A_{afn}(i-1,j,y) + S(x[i] \to \varepsilon) \mid y \in \{R,D,I\}\}$, else.~~

$$A_{afn}(i,j,D) = \begin{cases} \infty & , i = 0 \\ \min\{A_{afn}(i-1,j,D) + S(x[i] \to \varepsilon)\} \cup \\ \{A_{afn}(i-1,j,x) + g + S(x[i] \to c) \mid x \in \{R,I\}\} & , \text{else} \end{cases}$$

$$A_{afn}(i,j,I) = \begin{cases} \infty & , j = 0 \\ \min\{A_{afn}(i,j-1,I) + S(\varepsilon \to v[j]) \cup \\ A_{afn}(i,j-1,x) + g + S(\varepsilon \to v[j]) \mid x \in \{R,D\}\} & , \text{else} \end{cases}$$

cost of optimal affine gap cost alignment:

$$\min\{A_{afn}(m,n,x) \mid x \in \{R,D,I\}\}$$

Space $O(mn)$ Time $O(mn)$

↳ Bad 3x 6g